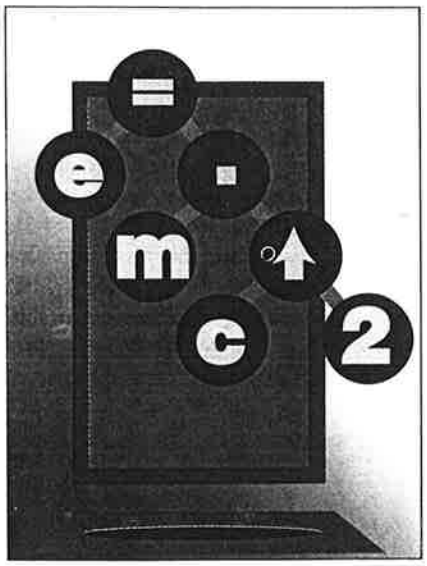
Joey Rogers

An Object-Oriented Expression Evaluator

If you have to write yet another expression evaluator, maybe this time you should start with a set of classes that does most of the hard bits.



Introduction

Evaluating expressions is a common operation in programming languages, so much so that many programmers may not appreciate the complexity of implementing an expression evaluator — that is, until they need to implement one of their own. Arithmetic and Boolean operator precedence, variable data, and infix notation all complicate the process of expression evaluation. It's tempting to try to simplify the implementation, by restricting it to a minimal set of operators, or disallowing parenthesis and variables. We could even do away with operator precedence by requiring users to enter expressions in postfix or prefix notation. Unless they are old FORTH programmers, however, they will probably rebel. Few users will want to change their way of thinking to accommodate the requirements of an application.

In this article, I present a set of expression objects that will properly implement a flexible and expandable infix expression evaluator. Sample uses include creating interfaces that allow users to enter values as expressions or as spreadsheet-like operations. The expression objects could even be used to implement a simple programming language interpreter. For those interested in object-oriented programming, this article also demonstrates an application for which inheritance and polymorphism are a natural fit.

Joey Rogers is an Application Engineer for the Intergraph Corporation in Huntsville, Alabama. His expertise includes neural networks, language interpretation, and object-oriented programming. He can be reached at srogers@hq.pcmail.ingr.com.

Design Specs

I need to set a few design boundaries before developing the expression evaluator. Specifically, the evaluator will:

- evaluate expressions in algebraic infix notation
- provide the following basic set of functionality:
  - Arithmetic operations (+, -, *, /)
  - Boolean operations (and, or, not)
  - Relational operations (>, <, >=, <=, ==, !=)
- handle constant as well as variable data maintained by a symbol table
- not be bounded by syntax
- be designed for expandability

While the above cannot fill the needs of every application needing an expression evaluator, it does cover a wide range of commonly needed capabilities.

Designing an Expression Tree

To begin the design process, I determine how an expression can be decomposed into objects. As shown in Figure 1, an expression, such as 1+2*3/(4-2), can be represented in a tree structure (known as an expression tree) which breaks an expression down into its components.

An in-order traversal of the expression tree will reconstruct the infix expression. Each node in the tree possesses a "value," which is either inherent (as in literals) or is some function of the values of its children (as in operators like +, –, *, /). Evaluating the root node (the + in Figure 1) will cause a post-order traversal of the expression tree, resulting in the value of the expression as a whole.

Base Classes

Beginning with the Exp_Obj class shown below, I begin modeling the nodes in the expression tree with objects.

```
// Base Expression Object

class Exp_Obj
  {
  public:
    virtual float
    GetValue( Symbol_Table &sym )
      {return 0.0;};
    virtual int GetType( void )
      {return NONE;};
    virtual int GetPriority( void )
      {return 0;};
  };
```

This class does nothing but define a standard interface for the rest of the expression objects. By deriving all other expression objects directly or indirectly from the base Exp_Obj class, I can use polymorphism and dynamic binding to facilitate easy manipulation of the expression objects, regardless of their specific type or functionality. The GetValue operation returns the value of the expression object. The derived classes will implement the functionality of the object by redefining this operation. The sym parameter to GetObject provides access to the symbol table information if needed. (I describe the symbol table in more detail later.)

I have defined five categories, or types, of expression objects: NONE,

OPERAND_TYPE, BINARY_OPERATOR_TYPE, UNARY_OPERATOR_TYPE, and EXPRESSION_TYPE. The GetType operation returns an expression object's type.

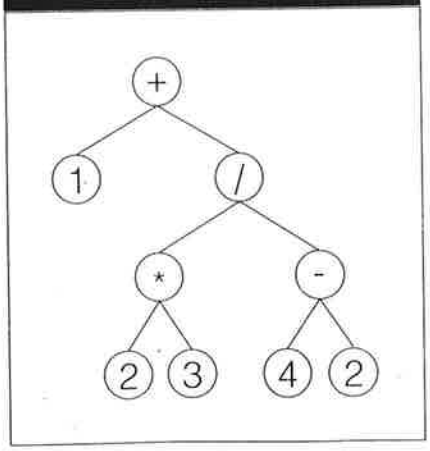
GetPriority returns the infix priority of an object, which is used to maintain operator precedence when evaluating the expression. In the expression 1+2*3/(4-2), the parentheses have the highest precedence, so their content is evaluated first. The * and / have equal precedence in this expression, so they are evaluated next, in a left-to-right order. The + operator has the lowest priority in this expression and is evaluated last. Table 1 shows the infix priority of the operators I implement as expression objects.

I derive the Unary_Obj (shown below) and Binary_Obj classes from Exp_Obj to abstract characteristics common to two types of operators: those that operate on a single object and those that operate on two objects. The Unary_Obj class holds a pointer to another Exp_Obj. Unary_Obj's constructors initialize this pointer, either to a NULL or to point to an Exp_Obj. SetObj enables changing the pointer after construction, and GetType always returns UNARY_OPERATOR_TYPE. Classes derived from Unary_Obj will redefine the pure virtual function GetValue to incorporate the functionality of the specific unary operator.

```
class Unary_Obj : public Exp_Obj
  {
  protected:
      Exp_Obj *obj;
  public:

    Unary_Obj( void ) { obj=NULL; }

    Unary_Obj( Exp_Obj *o  )
      { obj=o; }

    virtual void SetObj(Exp_Obj *o)
      { obj=o; }

    virtual int GetType( void )
      {return UNARY_OPERATOR_TYPE;}

    virtual float
    GetValue( Symbol_Table &sym )=0;
  };
```

The Binary_Obj class (below) is very similar to the Unary_Obj class. A Binary_Obj maintains pointers to both a left and a right Exp_Obj. The pointers are initialized at construction and may be changed thereafter via the SetLeft and SetRight operators. GetType always returns BINARY_OPERATOR_TYPE, and the pure virtual function GetValue is defined by derived classes to implement the binary operator's functionality.

Figure 1: *Expression Tree*



The EOS Kernel
Scalability in Real-Time
The ideal solution for real-time resource critical deeply embedded applications with the power of a totally scalable multi-tasking operating system.
A portable, configurable and customizable real-time environment that adapts to application architectures.
A complete offer of technology and services to effectively improve time-to-market.
ETNOTEAM Software Technologies
Via Adelaide Bono Cairoli, 34 - 20127 Milan - ITALY
tel.+39-2-26162-1, fax +39-2-26110755
Internet: http://www.etnoteam.it/
E-mail: products@etnoteam.it
Request Reader Service #134

```
class Binary_Obj : public Exp_Obj
  {
  protected:
    Exp_Obj *left, *right;
  public:

    Binary_Obj( void )
      { left=NULL; right=NULL; }

    Binary_Obj( Exp_Obj *l,
                Exp_Obj *r )
      { left=l; right=r; }

    virtual void
    SetLeft(Exp_Obj *l){ left=l;}

    virtual void
    SetRight(Exp_Obj *r){right=r;}

    virtual int GetType( void )
      {return BINARY_OPERATOR_TYPE;}
    virtual float
    GetValue( Symbol_Table &sym )=0;
  };
```

Operators

Now that I've provided for the basic structure of the expression tree, I can start to fill the tree with more concrete objects. Operators can connect to terminal objects (which may be either variables, such as A, or literals, such as 123.4) or to subtrees.

The Unary_Minus_Obj class implements the unary minus operation (-). It is derived from Unary_Obj. GetValue returns the negative of the referenced Exp_Obj's value, and GetPriority returns the operator's precedence. Remember that GetType and SetObj are also available since they are inherited from the Unary_Obj class.

```
class Unary_Minus_Obj : public
Unary_Obj
  {
  public:

    Unary_Minus_Obj( Exp_Obj *o ):
      Unary_Obj( o ){};

    Unary_Minus_Obj( void ) :
      Unary_Obj() {};

    virtual float
    GetValue(Symbol_Table &sym)
      {return -(obj->GetValue(sym));}
    virtual GetPriority( void )
      { return UNARY_SUB_PRIORITY; }
  };
```

The Add_Obj implements a binary addition (+) in its GetValue operation and returns the operator's infix priority in GetPriority. All children of Binary_Obj inherit SetLeft, SetRight, and GetType. The subtraction (-), multiplication (*), and division (/) operations are implemented in a similar fashion. The complete set of expression objects are available on the code disk and online sources.

```
class Add_Obj : public Binary_Obj
  {
  public:

    Add_Obj(Exp_Obj *l,Exp_Obj *r):
      Binary_Obj(l,r){};

    Add_Obj( void ):Binary_Obj(){};

    virtual int
    GetValue( Symbol_Table &sym )
      {return left->GetValue(sym) +
              right->GetValue(sym);}
    virtual int GetPriority( void )
      { return ADD_SUB_PRIORITY;}
  };
```

Terminal Objects

Literals and variables are what I call *terminals*; they occupy the leaves of the expression tree. I model literals with the

Table 1: *Operator Priority*

Operator	Priority
()	9
not	8
-	7
* /	6
+ -	5
< <= > >=	4
= <>	3
and	2
or	1

For Borland C/C++, Microsoft C/C++, Borland Pascal

RTKernel

Real-Time Multitasking for DOS

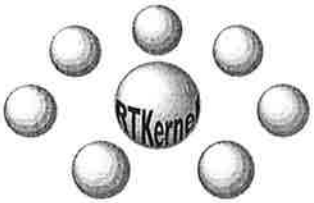
RTKernel is a professional, high-performance, real-time multitasking system for **MS-DOS** and **Embedded Systems**. It can use DOS device drivers and BIOS, and runs other DOS applications as a task - even **Windows!**

RTKernel is **loaded with features**: an unlimited number of tasks, excellent performance, a full set of inter-task communication functions (semaphores, mailboxes, synchronous message-passing), real and protected mode support, drivers for up to 38 COM ports and Novell's IPX services, and lots more...

It's **ROMable** and **very compact** (about 16K code, 6K data), making it ideally suited for Embedded Systems.

RTKernel is well-documented and easy to use. All hardware drivers always come with source code; kernel **source code available** at extra charge. **No run-time royalties.**

Join thousands of satisfied customers!



Use **RTKernel** for:
- process control
- data acquisition
- real-time simulations
- background processing

Libraries: $495
Source Code: add $445

In North America, please contact:
On Time Marketing
88 Christian Avenue · Setauket, New York 11733 · USA
Phone (516) 689-6654 · Fax (516) 689-1172
BBS (516) 689-6285 · FaxFacts (516) 689-6315 · CIS 73313,3177

From other countries, please contact:
On Time Marketing
Karolinenstrasse 32 · 20357 Hamburg · GERMANY
Phone +49-40-437472 · Fax +49-40-435196
CompuServe 100140,633

On Time MARKETING
Professional Programming Tools

Free demo disk! Request Info Kit C
Internet: http://www.on-time.com ftp.on-time.com info@on-time.com

Request Reader Service #135

Literal_Obj class, which defines a variable to hold the literal value. The literal's value is initialized by the constructor, GetType returns OPERAND_TYPE, and GetValue returns the literal value stored in the object. To simplify the interface to all objects, GetValue always takes a reference to the symbol table even when it is not needed.

```
class Literal_Obj : public Exp_Obj
  {
  private:
    float value;
  public:
    Literal_Obj( int v ) { value=v;}

    virtual float
    GetValue( Symbol_Table &sym )
      { return value; }

    virtual int GetType( void )
      { return OPERAND_TYPE; }
  };
```

The last component of the expression tree is the Variable_Obj, which holds not the variable's data, but its name. The variables' data resides in a symbol table. The symbol table maintains the name and value of each variable and provides access to each variable's data. The Symbol_Table class in Listing 1 provides three operations: Get, Set, and Is_In. The Get operation retrieves a variable's data by name, Set updates a variable's data by name (or creates the variable if it is not in the table), and Is_In returns a pointer to an entry in the symbol table if the variable exists. In the Variable_Obj class, the GetValue operation simply looks up the variable's value in the symbol table.

```
class Variable_Obj : public Exp_Obj
  {
  private:
    char name[25];
  public:

    Variable_Obj( char *var_name )
      { strcpy(name,var_name); }

    virtual float
    GetValue( Symbol_Table &sym )
      { return sym.Get(name); }

    virtual int GetType( void )
      { return OPERAND_TYPE; }
  };
```

Putting Together the Tree

The expression evaluator needs to find the expression objects arranged according to their associated expression tree. The process of linking the expression objects together to form the expression tree is the most complicated part of the Object-Oriented Expression Evaluator. Listing 2 shows part of a new class, Base_Expression, derived from Exp_Obj to build an expression tree from an expression string.

In Base_Expression the Exp_Obj pointer exp points to the root of the expression

Unique New Technology Guarantees Unequaled Benefits for Windows™ and NetWare® Programmers!

Compress your Windows .EXEs and .DLLs with Blinker 4.0!

Blinker 4.0 compresses Windows and DOS extended .EXEs and .DLLs, creates 32 bit Windows .EXEs for Windows NT and Windows '95 and creates .DLLs for DOS extended programs so you can share code and data between applications.

No other product on the market offers Windows & DOS extended .EXE and .DLL compression, which minimizes the costs of physical and electronic distribution of your programs, greatly reduces network traffic and encrypts your programs so they cannot be decompiled or disassembled!

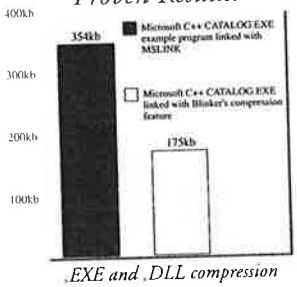
Blinker feature checklist:

- unique ability to compress Windows and DOS extended .EXEs and .DLLs by 50% or more
- Windows and DOS hosted linker to create .DLLs in DOS extended programs
- fastest available 16 & 32 bit Windows linker
- DOS extender to directly access up to 16Mb of memory, fully compatible with DPMI, VCPI and XMS programming standards and featuring a comprehensive API
- dynamic overlay linker to overcome the 640Kb conventional memory limit featuring EMS/XMS overlay caching for optimum execution speed
- unique dual mode feature to run the same .EXE automatically in either real or protected mode
- no runtime royalties
- a memory efficient super "SPAWN" command to run large programs, such as a word processor or a report generator, from inside your main program with less than a 10Kb memory overhead
- Microsoft C/C++, FORTRAN, Pascal, Assembler, Borland C/C++, Assembler, Symantec C++, Watcom C/C++ and CA-Clipper support
- full support for CodeView, Soft-ICE and Periscope debuggers
- serial number encryption
- creation of demonstration versions
- over 500 pages of documentation and a comprehensive on-line reference

Over 60,000 Users

.EXEs and .DLLs compressed by Blinker execute directly so you do not have to go through any decompression process to run them.

Proven Results!



.EXE and .DLL compression

The fastest Windows & DOS linker and a royalty-free DOS extender!!

Available in December 1995, Blinker 4.0 is also a royalty-free DOS extender, the world's fastest Windows linker *and* an award-winning dynamic overlay linker, all in one unique product for one low price! Compatible with C, C++, FORTRAN, Pascal, CA-Clipper and more, Blinker virtually eliminates "down time" with the fastest link times for Windows, DOS-extended and DOS programs.

Call today to see why over 60,000 programmers are already using this award-winning product!

The world's first network library for Visual Basic, Delphi, C/C ++ and CA-VO!!

Make your Visual Basic, Delphi, C/C++ and CA-VO programs NetWare-aware the easy way, with the latest version of this award-winning network library. Available December 1995, NOVLIB 3.0 features over 450 functions, written in highly optimized C and ASM code, for printer spooling, security, mapping, messaging, IPX/SPX handling, accounting and much more. NOVLIB 3.0 is the only network library which brings the power of Novell to Windows, DOS extended and real mode DOS programs for *all* these languages. If you program for Novell networks, you need NOVLIB! Call Blink*inc* now for your free sample program.

Save $50!
Blinker 4.0 $249
NOVLIB 3.0 $249
TEL: 1-804-747-6700
FAX: 1-804-747-4200

Blinkinc
6001 W. Broad St., Richmond VA 23294
In Europe contact: Blinkinc Ltd.
TEL:+44 1222 712444 • FAX:+44 1222 700888

Graphics copyright Blink, Inc 1995. Blinker and NOVLIB are trademarks of ASM, Inc. All other trademarks acknowledged.

❑ Request Reader Service #136 ❑

tree. GetType returns EXPRESSION_TYPE, and GetExp returns the original expression string. Base_Expression provides two GetValue operations, both of which return the value of the root node in the expression tree. The first function is equivalent to the GetValue in any other expression object. The second function creates a temporary symbol table with nothing in it so that the user does not have to create a symbol table when evaluating expressions that do not require one.

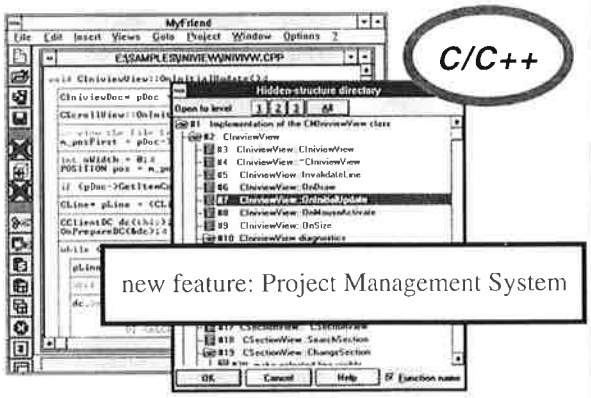
Building the expression tree involves four operations: Get_Token, Build_Expression, Build, and Push_Tree. The Get_Token operation finds the next token in the expression string and its type, and reports whether or not any errors occurred getting the token. Get_Token is a pure virtual function, which must be defined in a child class. Thus, the syntax of the expression (how the operators, identifiers, etc., are combined) is abstracted out of the basic objects. This gives the user a very flexible engine for implementing any number of expression syntax rules.

The Build_Expression operation creates the appropriate expression objects according to the information it receives from Get_Token and uses the Build operation to add the objects to the

Listing 1: *Symbol table class used to implement the variable data objects*

```
class Symbol_Table
    {
    private:
        struct entry { char name[25];    float value; entry *next;   };
        entry *head;
    public:
        Symbol_Table( void ) { head=NULL; };
        ~Symbol_Table( void )
            {
            entry *temp;
            while (head!=NULL) { temp=head; head=head->next;
                                 delete temp; };
            }
        entry *Is_In( char *name )
            {
            entry *temp=head;
            while (temp!=NULL)
                if (strcmp(name,temp->name)==0) return temp;
                else temp=temp->next;
            return NULL;
            }
        void Set( char *name, float value )
            {
            entry *temp=Is_In( name );
            if (temp==NULL)
                {
                entry *new_head=new entry;
                strcpy(new_head->name,name);
                new_head->value=value;
                new_head->next=head;
                head=new_head;
                }
            else temp->value=value;
            }
        float Get( char *name )
            {
            entry *temp=Is_In(name);
            if (temp!=NULL) return temp->value; else return 0;
            }
    };
/* End of File */
```


DON'T WORRY ABOUT EXPENSIVE SOFTWARE DOCUMENTATION...



... use *MyFriend*, the programming workbench for source code and NASSI-SHNEIDERMAN CHARTS.
Programming and documentation in one tool

MyFriend Release 2.0 $575 + shipping
Visa, MasterCard, Amex and Diners accepted

GSK
GSK GmbH
Unt. Pfaffensteigstr. 71
D-91126 Schwabach
Germany
Phone +49 911 63 75 85
FAX +49 911 63 75 86
BBS +49 911 63 24 968
CompuServe 100414,3044

Request Reader Service #137

68HC05/08/HC11/HC16/332
COSMIC C Tools Speed Development
617-932-2556
In Europe +33 (1) 43.99.53.90
COSMIC Software

Request Reader Service #138

expression tree. The Build_Expression and Build operations implement the algorithms shown in Figure 2.

The Build algorithm requires the use of two stacks: one for the expression being built and one to temporarily store operators. The ExpStack class shown in Listing 3 implements a stack data structure to store objects of the Exp_Obj type and provides three operations: Push, Pop, and Empty. Since all of the expression objects are derived directly or indirectly from Exp_Obj, polymorphism allows any expression object to be stored on the stack regardless of its specific type.

I provide four control objects to aid in the expression-tree building process: Open_Paren_Obj, Close_Paren_Obj, Start_Obj, and Stop_Obj (Listing 4). Open_Paren_Obj and Close_Paren_Obj supply a priority and type so they can be used in the Build operation to correctly override operator precedence when necessary. The Start and Stop objects identify the first and last tokens.

As objects are pushed onto the expression stack, the Push_Tree operation converts the stack into a tree as it is built. If Push_Tree

Listing 2: *Class* Base_Expression *points to root of expression tree*

```
class Base_Expression : Exp_Obj
    {
    protected:
        Exp_Obj *exp;
        ExpStack exp_stack, op_stack;
        // more protected elements ...
    public:
        Base_Expression( void ) { exp=NULL; }
        ~Base_Expression( void ) { delete exp; }
        virtual void GetToken(char *str,int &pos,char *buffer,
                              int &type, int &error)=0;
        virtual int GetType( void ) { return EXPRESSION_TYPE; }
        virtual float GetValue( Symbol_Table &sym )
            { return exp->GetValue(sym);}
        virtual float GetValue(void) {Symbol_Table sym;
            return exp->GetValue(sym);}
        virtual void Push_Tree(Exp_Obj *obj);
        virtual void Build( Exp_Obj *obj );
        virtual Exp_Obj *Build_Expression( char *str );
        // more member functions ...
        };
/* End of File */
```

Conversion Solutions for C and C++ Programmers

FOR_C® Converts standard FORTRAN plus VAX and many other extensions into ANSI C! Features C-style preprocessing for easy mapping of FORTRAN library calls, C prototyping for enhanced function call translations, basic static analysis and error checking, plus compiler-like performance. Translated code is extremely *readable* and *maintainable!*

FOR_C++® Converts legacy FORTRAN to readable C++ — for an even better representation of FORTRAN! Translations utilize C++ string and complex data classes, plus overloaded functions for better code readability. FOR_C++ generates C++ function prototypes, checks function calls for consistent usage and optimizes code for easy maintenance.

Different versions available to meet your project size. Call today for more information—and ask about our FORTRAN maintenance tools!

COBALT BLUE

COBALT BLUE, INC.
555 SUN VALLEY DRIVE, SUITE K-4
ROSWELL, GA 30076, USA
TEL (770) 518-1116, FAX (770) 640-1182
http://www.cobalt-blue.com

Request Reader Service #139

Figure 2: *Pseudo-code for tree construction*

```
Build_Expression

1. Push a Start_Obj onto the Operator Stack
2. For each expression object in the expression string (from left to
   right)
      a. Create an expression object according to the current
         token information
      b. Build( newly created expression object )

Build( Current Expression Object )

1. If the Current Expression Object is of the OPERAND_TYPE,
      Push the object onto the Expression Stack.
   Else If the Current Expression Object's type is
   BINARY_OPERAND_TYPE, UNARY_OPERAND_TYPE, or OPEN_PAREN_TYPE
      a. Pop an object off the Operator Stack and store it in
         Operator_Obj
      b. If Operator_Obj's type is OPEN_PAREN_TYPE,
            Priority1 = 0
         Else If Operator_Obj's type is CLOSE_PAREN_TYPE,
            Priority1 = -1;
         Else Priority1 = Operator_Obj's Priority
      c. Priority2 = Current Expression Object's Priority
      d. While Priority1 >= Priority2
            Push Operator_Obj onto Expression Stack
            Repeat Steps 1a and 1b
      e. Push Operator_Obj onto Operator Stack
      f. Push Current Expression Object onto Operator Stack
   Else If Current Expression Object is CLOSE_PAREN_TYPE,
      Until an object with the OPEN_PAREN_TYPE is reached,
         Pop objects off the Operator Stack and Push them onto
         the Expression Stack
   Else If Current Expression Object is STOP_TYPE
      Pop the remaining objects off of the Operand Stack and
      push them onto the Expression Stack.
```

Listing 3: *Stack to hold expression objects*

```
class ExpStack
    {
    private:
        struct node
            {
            Exp_Obj *data;
            node *next;
            };
        node *head;
    public:
```

Listing 3: *continued*

```
        ExpStack( void ) { head=NULL; };
        ~ExpStack( void )
            {
            node *temp;
            while (head!=NULL)
                { temp=head;    head=head->next;   delete temp; }
            }

        void Push(Exp_Obj *obj)
            {
            node *temp=new node;
            temp->data=obj;  temp->next=head;
            head=temp;
            };

        Exp_Obj *Pop( void )
            {
            if (head==NULL)
                {
                cout << "Stack Underflow...\n";
                exit(1);
                }
            node *temp=head;
            head=head->next;
            Exp_Obj *tempData=temp->data;
            delete temp;
            return tempData;
            };

        int Empty( void ) { if (head==NULL) return 1; else return 0; }
    };
/* End of File */
```

Figure 3: *Building an Expression Tree from an Expression String*

Expression String	Expression Stack	Operator Stack
1+2*3/(4-2)		
+2*3/(4-2)	1	
2*3/(4-2)	1	+
*3/(4-2)	1 2	+
3/(4-2)	1 2	+*
/(4-2)	1 2 3	+*
(4-2)	* 1 2 3	+/
4-2)	* 1 2 3	+/(
-2)	* 1 2 3 4	+/(
2)	* 1 2 3 4	+/(-
)	* 1 2 3 4 2	+/(-
	* - 1 2 3 4 2	+/
	/ * - 1 2 3 4 2	+
	+ / * - 1 2 3 4 2	

C-Index/II

Database Library

Shrink Wrap
Mission Critical
Cross-Platform

Used by leading developers in their award-winning software:
Intuit Symantec Banyan Stratus Boeing Mindscape Citicorp Timberline FDIC PG&E Computer Associates Leica JPL

C-Index/II is a C/C++ database library that stores and retrieves applicaiton information.

Complete information is available at our web site. To receive the free C-Index/II demo disk packet by mail, contact us by phone, fax or email.

For a detailed analysis of how C-Index/II will meet your needs, call or write with your specific requirements. A knowledgable developer will assist you.

- Windows 3.x, Win95, NT
- Unix Macintosh DOS OS/2
- PharLap Embedded Systems
- Millions of copies in use daily
- Proven reliability since 1983
- All Major Compilers
- No Application Royalties
- Full Source Code Included
- Individual License: $695

Partial Function List

High Level

dbcreate	dadd	dfind
dbopen	dupdate	dseq
mclose	ddelete	dread

Low Level and Multi-User

cdupadd	imgopen	msetlock
cunqadd	imageback	mtstlock
cchange	imagerest	mclrlock
cdelete	imgclose	msetsema
cfind	bcheck	mtstsema
cfirst	bbuild	mclrsema
clast	addroot	strtwrit
cnext	droproot	endwrit
cprev	cflush	cispeedon
cnextrep	cibufnum	cispeedoff

Exclusive Features

- Key Data
- ReadShare
- SpeedRead
- PowerFail Protection

Advanced Features

- Fast B+Tree Indexing
- Unlimited File Size and Key Types
- Complete Portable C Source Code
- Any system with ANSI or K&R C
- Live Cross Platform Data Access
- Multi-User and Single-User Access
- Data and Indexes in Same File
- Multiple Record Formats per File
- Link Directly to App or use DLL
- Over 80 API Functions
- Nine Simple High-Level Functions
- Very Flexible Low-Level Functions

Trio Systems
936 E. Green St. Suite 105
Pasadena, CA 91106
818/584-9706
818/584-0364 Fax
mail@triosystems.com
http://www.triosystems.com

Request Reader Service #140

is adding a unary operator to the tree, it will pop an object off the expression stack, set the operator object's pointer to the popped object, and push the operator object onto the expression stack. If a binary operator is added to the tree, Push_Tree will pop two objects off the expression stack, assign the operator object's left and right pointers to the two objects, and push the binary operator object onto expression stack. The only object left in the stack after the expression string is parsed will be the root of the expression tree. Figure 3 illustrates how an expression tree is created from an expression string using the algorithm described above.

One more class needs to be implemented before the expression objects can be used. Since the Get_Token operation is a pure virtual function in Base_Expression, a child class must be derived to implement it. I have created two classes; the C_Like_Expression class implements the Get_Token operation to recognize a C-like expression syntax while the Pascal_Like_Expression class implements a Pascal-like expression syntax. These classes are not shown here, but are available on the code disk and online sources. Listing 5 shows several examples utilizing the expression classes to evaluate both C-like and Pascal-like expressions.

Listing 4: *Control objects*

```
// Control Objects

class Start_Obj : public Exp_Obj
    {
    public:
        virtual int GetPriority( void ) { return 0; }
        virtual int GetType( void ) { return START_TYPE; }
    };
//- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
class Stop_Obj : public Exp_Obj
    {
    public:
        virtual int GetType( void ) { return STOP_TYPE; }
    };
//- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
class Open_Paren_Obj : public Exp_Obj
    {
    public:
        virtual int GetPriority( void ) { return OPEN_PAREN_PRIORITY; }
        virtual int GetType( void ) { return OPEN_PAREN_TYPE; }
    };
//- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
class Close_Paren_Obj : public Exp_Obj
    {
    public:
        virtual int GetPriority( void ) { return CLOSE_PAREN_PRIORITY; }
        virtual int GetType( void ) { return CLOSE_PAREN_TYPE; }
    };
//End of File
```

Hierarchy at a Glance

Now that I've presented all the components, I show the complete class hierarchy for the expression evaluator in Figure 4. This is by no means a complete set of objects; however, it is a strong foundation and allows easy addition of new objects. It is fairly simple to add support for different data types (like int, float, double, and long) into the expression objects.

Command-Line Evaluator

The code disk also includes a program that uses the C_Like_Expression class to implement a command-line expression evaluator. This example is very simple, but it illustrates how creating an Object-Oriented Expression Evaluator makes evaluating expressions easy.

Listing 5: *Sample use of expression evaluator*

```
C_Like_Expression c2("1 + 3 == 8/2 && 0");
cout << c2.GetExp() << " => "  << c2.GetValue()<< endl << endl;

Pascal_Like_Expression p2("1 + 3 = 8/2 and 0");
cout << p2.GetExp() << " => "  << p2.GetValue()<< endl << endl;

Symbol_Table sym;
sym.Set("v1",5); sym.Set("v2",9); sym.Set("v3",32);
sym.Set("Temperature",50);

C_Like_Expression c3("Temperature*v1/v2+v3");
cout << c3.GetExp() << " => "  << c3.GetValue(sym)<< endl << endl;

C_Like_Expression c4("1 && (1==v1 || !(4-1-2-1))");
cout << c4.GetExp() << " => "  << c4.GetValue(sym)<< endl << endl;

sym.Set("TRUE",1);
Pascal_Like_Expression p4("TRUE and (1=v1 or not(4-1-2-1))");
cout << p4.GetExp() << " => "  << p4.GetValue(sym)<< endl << endl;

//End of File

Output:

3. 1 + 3 == 8/2 && 0 => 0

4. 1 + 3 = 8/2 and 0 => 0

Temperature*v1/v2+v3 => 59.7778

1 && (1==v1 || !(4-1-2-1)) => 1

TRUE AND (1=v1 or NOT(4-1-2-1)) => 1
```

C and C++ DOCUMENTATION

C-METRIC™ ($59) – Complexity/Quality
• Calculates "cyclomatic" path complexity for functions and system
• Counts lines with comments, code, and 'C' statements

C-CALL™ ($69) – Function Hierarchy
• Tree-Diagram showing function hierarchy
• Table-of-Contents of functions versus files
• Summary and detailed cross-reference of functions

C-CMT™ ($69) – Function Comment
• Generates and inserts function comment blocks
• Can be re-run to update the comment blocks

C-LIST™ ($69) – Lists or Reformats
• Action-Diagrams show logic/control flow
• Reformats source to various standardized formats

C-REF™ ($69) – Cross-References Identifiers
• Local/global/define/parameter summary or cross-reference
• Produces class-hierarchy tree-diagram for C++ classes

C-BROWSE™ ($free in C-DOC) – Windows Tree Viewer
• Graphically view C-CALL function-trees or C-REF class-trees

C-DOC™ ($199) – DOS/Windows Package ($395 value)
• All 5 programs integrated as 1 overall C-DOC program
• Processes multiple directories/files up to 10,000 lines
• Unconditional 30-day money-back guarantee of satisfaction

C-DOC™ Professional ($299) – DOS, Windows, OS/2
• All features of C-DOC, processes 1,000,000 lines, 3-ring binder/case

!! NEW 6.0 !! Windows Graphic-Tree Viewer

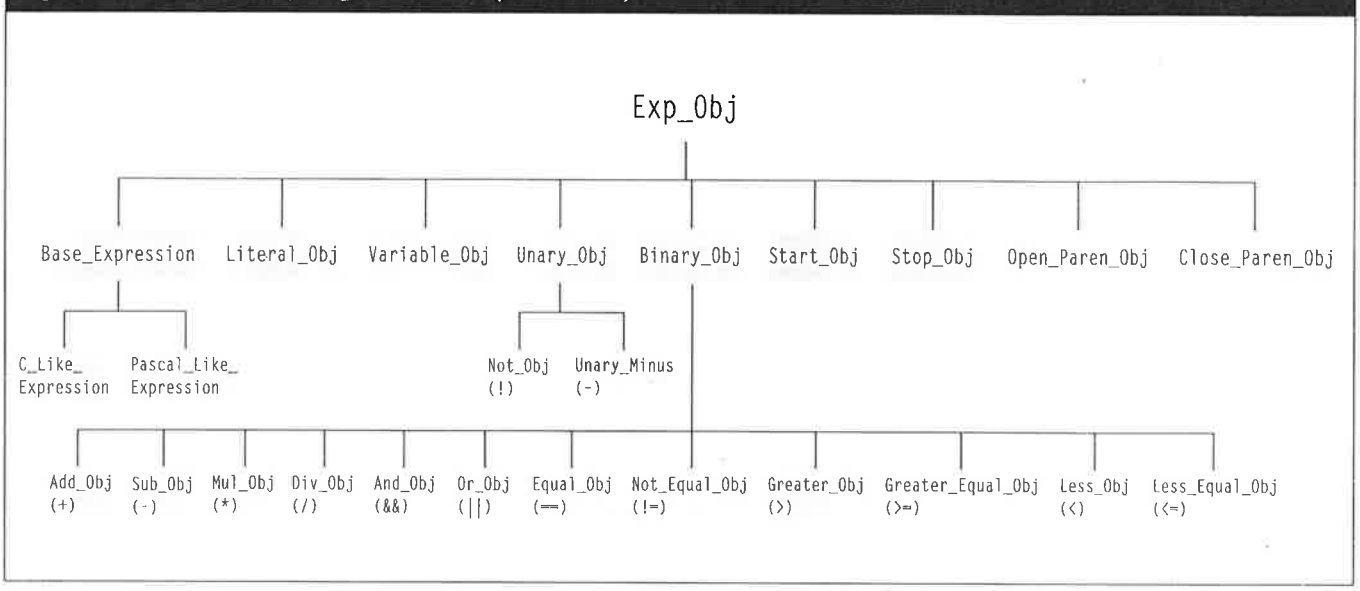
SOFTWARE BLACKSMITHS INC.
6064 St Ives Way
Mississauga ONT
Canada L5N-4M1
Voice/Fax (905) 858-4466
http://swbs.idirect.com
VISA MasterCard
CALL NOW!

Request Reader Service #141

Conclusion

C++'s inheritance and polymorphism features are especially effective in modeling the semantics of infix algebraic expressions. The Object-Oriented Expression Evaluator provides the programmer with a set of objects that exploit the features of an object-oriented programming language to create objects that are easy to understand, flexible, and expandable. Using the hierarchy presented here as a starting point, it should be possible to create code to handle more elaborate and interesting kinds of expressions. ❑

Figure 4: *Class hierarchy diagram for the Expression Objects*



PL/1 to C/C++ Code Translator
PL/1 Static Analyzer

This translator truly understands PL/1. It will cope with the idiosyncrasies which make this old language unique.

- De-nests PL/1 PROCS - *Internal symbol tables* resolve identifier scope by extending function declarations with required parameters.
- *C++ classes* emulate PL/1 types : CHAR array, PIC, BIT string..
- Supports *PL/1 structure declarations* across procs, resolves incomplete structure uses, moves indices to their proper location.
- Supports most *builtins* contextually (SUBSTR, STRING)...

Designed from the ground up to help you port your PL/1 legacy code to C/C++ or other languages.

- It will generate practically *error free compile time C/C++* with plenty of support to help you in your migration effort.
- Our optional **PL/1 Static Analyzer** will help you evaluate the cost of migrating your code. It counts tokens, prints out symbol tables with full declaration details, statements types, does metrics...

But don't just take our word for it, ask for our full-featured demo version.

Call or email for our white paper on PL/1 translation issues.

USA and Canada :
Solar Computer Exchange, Inc. Phone (508)872-7298
18 Main St., Framingham, Mass. 01701, USA FAX (508)879-6782
Compuserve 100126,3521
Other countries : Internet 100126.3521@compuserve.com
Micro Sigma, S. A. Phone +33-1-46.22.99.88
42, rue Legendre, 75017 Paris - France FAX +33-1-47.63.84.42

❑ Request Reader Service #142 ❑

Picasso

Create Windows interfaces for legacy applications in minutes, not months!

Picasso enables the programmer to capture legacy screens and map new controls to legacy functions. **Picasso** generates ready-to-run Visual Basic front ends for IBM **3270/5250**, Siemens **9750** and **DOS** applications. No modification of legacy source code is required. **Picasso** is wrapper technology at its best!

WINGate Tech. • Morristown NJ, 07960, USA
Tel 800.946.4283 & 201.539.2727 • Fax 201.539.2838
wingate@wingate.com • http://www.wingate.com
INTRA-SYS • Hannover, Germany • Tel 0511.61389.0 • Fax 0511.61389.99 • picasso@intra-sys.de

❑ Request Reader Service #143 ❑